Panoptic audit details

- Total Prize Pool: $56,000 in USDC
  - HM awards: up to $50,400 in USDC
    - If no valid Highs or Mediums are found, the HM pool is $0
  - QA awards: $2,100 in USDC
  - Judge awards: $3,000 in USDC
  - Scout awards: $500 in USDC
- Read our guidelines for more details
- Starts December 19, 2025 20:00 UTC
- Ends January 7, 2026 20:00 UTC

❗ Important notes for wardens

1. A coded, runnable PoC is required for all High/Medium submissions to this audit.
   - This repo includes a basic template to run the test suite.
   - PoCs must use the test suite provided in this repo.
   - Your submission will be marked as Insufficient if the POC is not runnable and working with the provided test suite.
   - Exception: PoC is optional (though recommended) for wardens with signal ≥ 0.68.
2. Judging phase risk adjustments (upgrades/downgrades):
   - High- or Medium-risk submissions downgraded by the judge to Low-risk (QA) will be ineligible for awards.
   - Upgrading a Low-risk finding from a QA report to a Medium- or High-risk finding is not supported.
   - As such, wardens are encouraged to select the appropriate risk level carefully during the submission phase.

V12 findings

V12 is Zellic's in-house AI auditing tool. It is the only autonomous Solidity auditor that reliably finds Highs and Criticals. All issues found by V12 will be judged as out of scope and ineligible for awards.

V12 findings will be posted in this section within the first two days of the competition.

Links

- **Previous audits:**
  - Obsidian Audits
  - Nethermind
- **Documentation:**
  - https://docs.panoptic.xyz/
  - Litepaper
  - Whitepaper
  - Blog
  - YouTube

- **Website:** https://www.panoptic.xyz
- **X/Twitter:** https://twitter.com/Panoptic_xyz
- **Codebase Walkthrough:** https://www.youtube.com/watch?v=Qsre4XiD_t4
- **Original Repo**: https://github.com/panoptic-labs/panoptic-v1-core
- **Further Reading**: Panoptic has been presented at conferences and was conceived with the first Panoptic's Genesis blog post in mid-summer 2021:
    - Panoptic @ EthCC 2023
    - Panoptic @ ETH Denver 2023
    - Panoptic @ ETH Denver 2022
    - Panoptic @ DeFi Guild
    - Panoptic's Genesis: Blog Series

Publicly known issues

Anything included in this section and its subsection is considered a publicly known issue and is therefore ineligible for awards.

System & Token Limitations

- Transfers of ERC1155 SFPM tokens has been disabled.
- Construction helper functions (prefixed with add) in the TokenId library and other types do not perform extensive input validation. Passing invalid or nonsensical inputs into these functions or attempting to overwrite already filled slots may yield unexpected or invalid results. This is by design, so it is expected that users of these functions will validate the inputs beforehand.
- Tokens with a supply exceeding 2^127 - 1 are not supported.
- If one token on a pool is broken/does not meet listed criteria/is malicious there are no guarantees as to the security of the other token in that pool, as long as other pools with two legitimate and compliant tokens are not affected.

Oracle & Price Manipulation

- Price/oracle manipulation that is not atomic or requires attackers to hold a price across more than one block is not in scope -i.e., to manipulate the internal exponential moving averages (EMAs), you need to set the manipulated price and then keep it there for at least 1 minute until it can be updated again.
- Attacks that stem from the EMA oracles being extremely stale compared to the market price within its period (currently between 2-30 minutes)
- As a general rule, only price manipulation issues that can be triggered by manipulating the price atomically from a normal pool/oracle state are valid

Protocol Parameters

- The constants VEGOID, EMA_PERIODS, MAX_TICKS_DELTA, MAX_TWAP_DELTA_LIQUIDATION, MAX_SPREAD, BP_DECREASE_BUFFER, MAX_CLAMP_DELTA, NOTIONAL_FEE, PREMIUM_FEE, PROTOCOL_SPLIT, BUILDER_SPLIT, SELLER_COLLATERAL_RATIO, BUYER_COLLATERAL_RATIO, MAINT_MARGIN_RATE, FORCE_EXERCISE_COST, TARGET_POOL_UTIL, SATURATED_POOL_UTIL, MAX_OPEN_LEGS, and the IRM parameters (CURVE_STEEPNESS, TARGET_UTILIZATION, etc.) are all parameters and subject to change, but within reasonable levels.

Premium & Liquidation Issues

- Given a small enough pool and low seller diversity, premium manipulation by swapping back and forth in Uniswap is a known risk. As long as it's not possible to do it between two of your own accounts profitably and doesn't cause protocol loss, that's acceptable
- It's known that liquidators sometimes have a limited capacity to force liquidations to execute at a less favorable price and extract some additional profit from that. This is acceptable even if it causes some amount of unnecessary protocol loss.
- It's possible to leverage the rounding direction to artificially inflate the total gross premium and significantly decrease the rate of premium option sellers earn/are able to withdraw (but not the premium buyers pay) in the future (only significant for very-low-decimal pools, since this must be done one token at a time).
- It's also possible for options buyers to avoid paying premium by calling settleLongPremium if the amount of premium owed is sufficiently small.
- Premium accumulation can become permanently capped if the accumulator exceeds the maximum value; this can happen if a low amount of liquidity earns a large amount of (token) fees

Gas & Execution Limitations

- The liquidator may not be able to execute a liquidation if MAX_POSITIONS is too high for the deployed chain due to an insufficient gas limit. This parameter is not final and will be adjusted by deployed chain such that the most expensive liquidation is well within a safe margin of the gas limit.
- It's expected that liquidators may have to sell options, perform force exercises, and deposit collateral to perform some liquidations. In some situations, the liquidation may not be profitable.
- In some situations (stale oracle tick), force exercised users will be worse off than if they had burnt their position.

Share Supply Issues

- It is feasible for the share supply of the CollateralTracker to approach 2**256 - 1 (given the token supply constraints, this can happen through repeated protocol-loss-causing liquidations), which can cause various reverts and overflows. Generally, issues with an extremely high share supply as a precondition (delegation reverts due to user's balance being too high, other DoS caused by overflows in calculations with share supply or balances, etc.) are not valid unless that share supply can be created through means other than repeated liquidations/high protocol loss.

Constructor Assumptions

- For the purposes of this competition, assume the constructor arguments to the RiskEngine are: 10_000_000, 10_000_000, address(0), address(0)

Out of Scope

- Front-running via insufficient slippage specification is not in scope

Additional Findings from Nethermind pre-contest

1. **Double Penalty / Index Update**

   Users pay the interest penalty even when using phantom shares for interest payment. In force exercise scenarios, the user pays in `delegate(...)` and their borrow index is also updated in `_accrueInterest`. In the regular penalty case, the index is not updated.

2. **Masking Insolvency Magnitude**

   While the else cases in `_getMargin(...)` correctly resolve the staleness issue, the if statement (where interest owed > balance) masks the true deficit magnitude. Since `_getMargin(...)` is used in `isAccountSolvent(...)`, setting interest (requirement) to the balance value hides the actual funds shortage.

   Example: Alice owes 100 interest with a balance of 20. Setting interest to 20 and balance to 0 shows a deficit of 20 instead of the actual deficit of 80.

3. **Broken Bonus Calculations**

   The if statement logic in `_getMargin(...)` breaks bonus calculations by hiding the true deficit. The values of bonus cross and threshold cross are calculated based on the masked deficit rather than the actual shortage.

4. **Orphan Shares in Delegate/Revoke**

   The `delegate(...)` to `revoke(...)` interaction creates shares not owned by anyone, breaking the supply invariant. In force exercise scenarios:

   1. User starts with balance X.
   2. **Delegate:** User balance inflates to inflation + X, then decrements by X due to insufficient interest payment. Balance = inflation.
   3. **Settle Burn / Accrue Interest:** Y shares are burned to cover interest (sufficient phantom balance). Total supply decreases by Y. User balance = inflation - Y.
   4. **Revoke:** Since inflation > balance, user balance is zeroed and total supply is restored by adding Y (inflation - (inflation - Y)).
   5. **Result:** Net change in total supply is 0 (-Y burn +Y restore). The original X shares remain in total supply but are owned by no one.

Overview

Panoptic is a permissionless options trading protocol. It enables the trading of perpetual options on top of any Uniswap V3 and Uniswap V4 pool.

The Panoptic protocol is noncustodial, has no counterparty risk, offers instantaneous settlement, and is designed to remain fully collateralized at all times.

Core Contracts

SemiFungiblePositionManager

A gas-efficient alternative to Uniswap's NonFungiblePositionManager that manages complex, multi-leg Uniswap positions encoded in ERC1155 tokenIds, performs swaps allowing users to mint positions with only one type of token, and, most crucially, supports the minting of both typical LP positions where liquidity is added to Uniswap and "long" positions where Uniswap liquidity is burnt. While the SFPM is enshrined as a core component of the protocol and we consider it to be the "engine" of Panoptic, it is also a public good

that we hope savvy Uniswap V3 and V4 LPs will grow to find an essential tool and upgrade for managing their liquidity.

RiskEngine

The central risk assessment and solvency calculator for the Panoptic Protocol. This contract serves as the mathematical framework for all risk-related calculations and does not hold funds or state regarding user balances. The RiskEngine is responsible for:

- **Collateral Requirements**: Calculating the required collateral for complex option strategies including spreads, strangles, iron condors, and synthetic positions based on position composition and pool utilization
- **Solvency Verification**: Determining whether an account meets the maintenance margin requirements through the `isAccountSolvent` function, accounting for cross-collateralization between token0 and token1
- **Liquidation Parameters**: Computing liquidation bonuses paid to liquidators and protocol loss via `getLiquidationBonus`, factoring in the account's token balances and position requirements
- **Force Exercise Costs**: Calculating the cost to forcefully exercise out-of-range long positions via `exerciseCost`, using an exponentially decaying function based on distance from strike
- **Adaptive Interest Rate Model**: Computing dynamic borrow rates based on pool utilization using a PID controller approach, with rates adjusting between minimum and maximum thresholds to target optimal utilization
- **Oracle Management**: Managing the internal pricing oracle with volatility safeguards, exponential moving averages (EMAs), and median filters to prevent price manipulation
- **Risk Parameters**: Storing and providing access to protocol-wide risk parameters including seller/buyer collateral ratios, commission fees, force exercise costs, and target pool utilization levels
- **Guardian Controls**: Enabling an authorized guardian address to override safe mode settings and lock/unlock pools in emergency situations

The RiskEngine uses sophisticated algorithms including utilization-based multipliers (modulated by the VEGOID parameter), cross-buffer ratios for cross-collateralization, and dynamic collateral requirements that scale with pool utilization to ensure protocol solvency at all times.

CollateralTracker

An ERC4626 vault where token liquidity from passive Panoptic Liquidity Providers (PLPs) and collateral for option positions are deposited. The CollateralTracker is responsible for:

- **Asset Management**: Tracking deposited assets, assets deployed in the AMM, and credited shares from long positions that exceed the rehypothecation threshold
- **Interest Accrual**: Implementing a compound interest model where borrowers (option sellers) pay interest on borrowed liquidity, with rates determined by the RiskEngine based on pool utilization
- **Commission Handling**: Collecting and distributing commission fees on option minting and burning, splitting fees between the protocol, builders (if a builder code is present), and PLPs
- **Premium Settlement**: Facilitating the payment and receipt of options premia between buyers and sellers, including settled and unsettled premia calculations
- **Balance Operations**: Managing user share balances through deposits, withdrawals, mints, redeems, and the delegation/revocation of virtual shares for active positions

- **Liquidation Settlement**: Handling the settlement of liquidation bonuses by minting shares to liquidators and managing protocol loss when positions are liquidated
- **Collateral Refunds**: Processing refunds between users when positions are closed, force-exercised, or adjusted

Each CollateralTracker maintains its own market state including a global borrow index for compound interest calculations, tracks per-user interest states (net borrows and last interaction snapshots), and coordinates with the RiskEngine to determine appropriate interest rates based on real-time pool utilization.

PanopticPool

The Panoptic Pool exposes the core functionality of the protocol. If the SFPM is the "engine" of Panoptic, the Panoptic Pool is the "conductor". All interactions with the protocol, be it minting or burning positions, liquidating or force exercising distressed accounts, or just checking position balances and accumulating premiums, originate in this contract. It is responsible for:

- **Position Orchestration**: Coordinating calls to the SFPM to create, modify, and close option positions in Uniswap
- **Premium Tracking**: Tracking user balances and accumulating premia on option positions over time
- **Solvency Checks**: Consulting the RiskEngine to verify account solvency before allowing position changes or withdrawals
- **Settlement Coordination**: Calling the CollateralTracker with the necessary data to settle position changes, including commission payments, interest accrual, and balance updates
- **Risk Validation**: Ensuring all operations comply with the risk parameters and collateral requirements calculated by the RiskEngine

Architecture & Actors

Each instance of the Panoptic protocol on a Uniswap pool contains:

- One PanopticPool that orchestrates all interactions in the protocol
- One RiskEngine that calculates collateral requirements, verifies solvency, and manages risk parameters
- Two CollateralTrackers, one for each constituent token0/token1 in the Uniswap pool
- A canonical SFPM - the SFPM manages liquidity across every Panoptic Pool

There are five primary roles assumed by actors in this Panoptic Ecosystem:

Panoptic Liquidity Providers (PLPs)

Users who deposit tokens into one or both CollateralTracker vaults. The liquidity deposited by these users is borrowed by option sellers to create their positions - their liquidity is what enables undercollateralized positions. In return, they receive commission fees on both the notional and intrinsic values of option positions when they are minted, as well as interest payments from borrowers. Note that options buyers and sellers are PLPs too - they must deposit collateral to open their positions. We consider users who deposit collateral but do not *trade* on Panoptic to be "passive" PLPs.

Option Sellers

These users deposit liquidity into the Uniswap pool through Panoptic, making it available for options buyers to remove. This role is similar to providing liquidity directly to Uniswap V3, but offers numerous benefits including advanced tools to manage risky, complex positions and a multiplier on the fees/premia generated by their liquidity when it is removed by option buyers. Option sellers pay interest to PLPs on borrowed liquidity, with rates dynamically adjusted by the RiskEngine based on pool utilization. Sold option positions on Panoptic have similar payoffs to traditional options.

Option Buyers

These users remove liquidity added by option sellers from the Uniswap Pool and move the tokens back into Panoptic. The premia they pay to sellers for the privilege is equivalent to the fees that would have been generated by the removed liquidity, plus a spread multiplier based on the portion of available liquidity in their Uniswap liquidity chunk that has been removed or utilized.

Liquidators

These users are responsible for liquidating distressed accounts that no longer meet the collateral requirements calculated by the RiskEngine. They provide the tokens necessary to close all positions in the distressed account and receive a bonus from the remaining collateral, calculated by the RiskEngine's liquidation bonus formula. Sometimes, they may also need to buy or sell options to allow lower liquidity positions to be exercised.

Force Exercisors

These are usually options sellers. They provide the required tokens and forcefully exercise long positions (from option buyers) in out-of-range strikes that are no longer generating premia, so the liquidity from those positions is added back to Uniswap and the sellers can exercise their positions (which involves burning that liquidity). They pay a fee to the exercised user for the inconvenience, with the fee amount determined by the RiskEngine's `exerciseCost` function.

Flow

All protocol users first onboard by depositing tokens into one or both CollateralTracker vaults and being issued shares (becoming PLPs in the process). Panoptic's CollateralTracker supports the full ERC4626 interface, making deposits and withdrawals a simple and standardized process. Passive PLPs stop here.

Once they have deposited, all interactions with the protocol are initiated through the PanopticPool's unified entry points:

- `dispatch()` - The primary entry point for users to execute actions on their own behalf
- `dispatchFrom()` - Allows approved operators to execute actions on behalf of another user

These entry points accept encoded action data that specifies the operation to perform, which can include:

- Minting option positions with up to four distinct legs, each encoded in a positionID/tokenID as either short (sold/added) or long (bought/removed) liquidity chunks. The RiskEngine verifies that the account will remain solvent after minting.
- Burning or exercising positions. The RiskEngine ensures collateral requirements are met during the burn process.
- Settling long premium to force solvent option buyers to pay any premium owed to sellers

- Poking the median oracle to insert a new observation into the RiskEngine's internal median ring buffer
- Force exercising out-of-range long positions held by other users, with costs calculated by the RiskEngine
- Liquidating distressed accounts that no longer meet collateral requirements, with bonuses determined by the RiskEngine

This unified dispatch architecture provides a consistent interface for all protocol interactions while allowing the PanopticPool to orchestrate the necessary calls to the SFPM, CollateralTracker, and RiskEngine based on the requested action.

Scope

See scope.txt

Files in scope

Note: The nSLoC counts in the following table have been automatically generated and may differ depending on the definition of what a "significant" line of code represents. As such, they should be considered indicative rather than absolute representations of the lines involved in each contract.

File	nSLOC
/contracts/PanopticPool.sol	1183
/contracts/RiskEngine.sol	1294
/contracts/types/OraclePack.sol	291
/contracts/types/MarketState.sol	65
/contracts/types/PoolData.sol	42
/contracts/types/RiskParameters.sol	72
/contracts/SemiFungiblePositionManager.sol	673
/contracts/SemiFungiblePositionManagerV4.sol	631
/contracts/CollateralTracker.sol	863
/contracts/libraries/PanopticMath.sol	369
/contracts/libraries/Math.sol	641
/contracts/types/TokenId.sol	232
Total	**6356**

The following contracts should have the diffs reviewed:

- SemiFungiblePositionManager.sol: SemiFungiblePositionManager.sol.diff
- SemiFungiblePositionManagerV4.sol: SemiFungiblePositionManagerV4.sol.diff
- CollateralTracker.sol: CollateralTracker.sol.diff

- libraries/PanopticMath.sol: PanopticMath.sol.diff
- libraries/Math.sol: Math.sol.diff
- types/TokenId.sol: TokenId.sol.diff

Files out of scope

See out_of_scope.txt

File
./contracts/PanopticFactory.sol
./contracts/PanopticFactoryV4.sol
./contracts/base/FactoryNFT.sol
./contracts/base/MetadataStore.sol
./contracts/base/Multicall.sol
./contracts/interfaces/IRiskEngine.sol
./contracts/interfaces/ISemiFungiblePositionManager.sol
./contracts/libraries/CallbackLib.sol
./contracts/libraries/Constants.sol
./contracts/libraries/EfficientHash.sol
./contracts/libraries/Errors.sol
./contracts/libraries/FeesCalc.sol
./contracts/libraries/InteractionHelper.sol
./contracts/libraries/SafeTransferLib.sol
./contracts/libraries/V4StateReader.sol
./contracts/tokens/ERC1155Minimal.sol
./contracts/tokens/ERC20Minimal.sol
./contracts/tokens/interfaces/IERC20Partial.sol
./contracts/types/LeftRight.sol
./contracts/types/LiquidityChunk.sol
./contracts/types/Pointer.sol
./contracts/types/PositionBalance.sol
./script/DeployProtocol.s.sol
./test/foundry/core/CollateralTracker.t.sol
./test/foundry/core/Misc.t.sol

File
./test/foundry/core/PanopticFactory.t.sol
./test/foundry/core/PanopticPool.t.sol
./test/foundry/core/RiskEngine/AssertExt.sol
./test/foundry/core/RiskEngine/RiskEngine.Gaps.t.sol
./test/foundry/core/RiskEngine/RiskEngine.Properties.t.sol
./test/foundry/core/RiskEngine/RiskEngine.Scenarios.t.sol
./test/foundry/core/RiskEngine/RiskEngineHarness.sol
./test/foundry/core/RiskEngine/RiskEngineIRM.t.sol
./test/foundry/core/RiskEngine/RiskEngineInvariants.t.sol
./test/foundry/core/RiskEngine/RiskEnginePropertiesPlus.t.sol
./test/foundry/core/RiskEngine/RiskEngineSafeModeAndOracle.t.sol
./test/foundry/core/RiskEngine/helpers/PositionFactory.sol
./test/foundry/core/RiskEngine/mocks/MockCollateralTracker.sol
./test/foundry/core/SemiFungiblePositionManager.t.sol
./test/foundry/coreV3/CollateralTracker.t.sol
./test/foundry/coreV3/Misc.t.sol
./test/foundry/coreV3/PanopticFactory.t.sol
./test/foundry/coreV3/PanopticPool.t.sol
./test/foundry/coreV3/RiskEngine/AssertExt.sol
./test/foundry/coreV3/RiskEngine/RiskEngine.Gaps.t.sol
./test/foundry/coreV3/RiskEngine/RiskEngine.Properties.t.sol
./test/foundry/coreV3/RiskEngine/RiskEngine.Scenarios.t.sol
./test/foundry/coreV3/RiskEngine/RiskEngineHarness.sol
./test/foundry/coreV3/RiskEngine/RiskEngineIRM.t.sol
./test/foundry/coreV3/RiskEngine/RiskEngineInvariants.t.sol
./test/foundry/coreV3/RiskEngine/RiskEnginePropertiesPlus.t.sol
./test/foundry/coreV3/RiskEngine/RiskEngineSafeModeAndOracle.t.sol
./test/foundry/coreV3/RiskEngine/helpers/PositionFactory.sol
./test/foundry/coreV3/RiskEngine/mocks/MockCollateralTracker.sol
./test/foundry/coreV3/SemiFungiblePositionManager.t.sol

File
./test/foundry/libraries/CallbackLib.t.sol
./test/foundry/libraries/FeesCalc.t.sol
./test/foundry/libraries/Math.t.sol
./test/foundry/libraries/PanopticMath.t.sol
./test/foundry/libraries/PositionAmountsTest.sol
./test/foundry/libraries/SafeTransferLib.t.sol
./test/foundry/libraries/harnesses/CallbackLibHarness.sol
./test/foundry/libraries/harnesses/FeesCalcHarness.sol
./test/foundry/libraries/harnesses/MathHarness.sol
./test/foundry/libraries/harnesses/PanopticMathHarness.sol
./test/foundry/testUtils/ERC20S.sol
./test/foundry/testUtils/PositionUtils.sol
./test/foundry/testUtils/PriceMocks.sol
./test/foundry/testUtils/ReentrancyMocks.sol
./test/foundry/testUtils/V4RouterSimple.sol
./test/foundry/test_periphery/PanopticHelper.sol
./test/foundry/tokens/ERC1155Minimal.t.sol
./test/foundry/types/LeftRight.t.sol
./test/foundry/types/LiquidityChunk.t.sol
./test/foundry/types/PositionBalance.t.sol
./test/foundry/types/TokenId.t.sol
./test/foundry/types/harnesses/LeftRightHarness.sol
./test/foundry/types/harnesses/LiquidityChunkHarness.sol
./test/foundry/types/harnesses/PositionBalanceHarness.sol
./test/foundry/types/harnesses/TokenIdHarness.sol

Totals: 80

Additional context

Areas of concern (where to focus for bugs)

The factory contract and usage of libraries by external integrators is relatively unimportant -- wardens should focus their efforts on the security of the SFPM, PanopticPool, RiskEngine, and CollateralTracker

Main invariants

SFPM (SemiFungiblePositionManager)

- Fees collected from Uniswap during any given operation should not exceed the amount of fees earned by the liquidity owned by the user performing the operation.
- Fees paid to a given user should not exceed the amount of fees earned by the liquidity owned by that user.

CollateralTracker

Asset Accounting

- `totalAssets()` must equal `s_depositedAssets + s_assetsInAMM + unrealizedGlobalInterest` at all times
- `totalSupply()` must equal `_internalSupply + s_creditedShares` at all times
- `s_depositedAssets` should never underflow below 1 (the initial virtual asset)
- The share price (`totalAssets() / totalSupply()`) must be non-decreasing over time (except for rounding in favor of the protocol and during liquidations with protocol loss)

Interest Accrual

- The global `borrowIndex` must be monotonically increasing over time and start at 1e18 (WAD)
- For any user with `netBorrows > 0`, their `userBorrowIndex` must be ≤ the current global `borrowIndex`
- Interest owed by a user must equal: `netBorrows * (currentBorrowIndex - userBorrowIndex) / userBorrowIndex`
- `unrealizedGlobalInterest` must never exceed the sum of all individual users' interest owed
- After `_accrueInterest()`, the user's `userBorrowIndex` must equal the current global `borrowIndex` (unless insolvent and unable to pay)

Credited Shares

- `s_creditedShares` represents shares for long positions and can only increase when positions are created and decrease when closed
- When a long position is closed, if `creditDelta > s_creditedShares`, the difference must be paid by the option owner as a rounding haircut
- The rounding haircut from Uniswap position management should not exceed a few wei per position

Deposits and Withdrawals

- Users with open positions (`numberOfLegs > 0`) cannot transfer shares via `transfer()` or `transferFrom()`
- Users with open positions can only withdraw if they provide `positionIdList` and remain solvent after withdrawal

- Deposits must not exceed `type(uint104).max` (2^104 - 1)
- Withdrawals must leave at least 1 asset in `s_depositedAssets` (cannot fully drain the pool)

RiskEngine

Solvency Checks

- An account is solvent if and only if: `balance0 + convert(scaledSurplus1) >= maintReq0` AND `balance1 + convert(scaledSurplus0) >= maintReq1` (where conversion direction depends on price)
- The maintenance requirement includes: position collateral requirements + accrued interest owed + long premia owed - short premia owed - credit amounts
- Cross-collateralization uses a `crossBufferRatio` that scales based on pool utilization (higher utilization = more conservative buffer)
- Solvency must be checked at the oracle tick, not the current tick, to prevent manipulation

Collateral Requirements

- Collateral requirements must scale with pool utilization (higher utilization = higher requirements)
- The "global utilization" used for margin calculations is the maximum utilization across all of a user's positions at the time they were minted

Liquidation Bonuses

- Liquidation bonus = `min(collateralBalance / 2, required − collateralBalance)`
- The bonus is computed cross-collaterally using both tokens converted to the same denomination
- If the liquidatee has insufficient balance in one token, excess balance in the other token can be used to mitigate protocol loss through token conversion
- If premium was paid to sellers during liquidation, it must be clawed back (haircut) if it would cause protocol loss to exceed the remaining collateral

Premium Haircutting

- If `collateralRemaining < 0` (protocol loss exists), premium paid to sellers during the liquidation must be proportionally clawed back
- The haircut is applied per-leg based on the ratio of protocol loss to premium paid
- After haircutting, the adjusted bonus must not result in protocol loss exceeding the initial collateral balance

Force Exercise Costs

- Base force exercise cost = 1.024% (`FORCE_EXERCISE_COST = 102_400 / 10_000_000`) of notional for in-range positions
- Cost for out-of-range positions = 1 bps (`ONE_BPS = 1000 / 10_000_000`)
- The cost must account for token deltas between current and oracle prices for all long legs
- Only long legs (not short legs) contribute to force exercise costs

Interest Rate Model

- Interest rate must be bounded: `MIN_RATE_AT_TARGET ≤ rate ≤ MAX_RATE_AT_TARGET`

- Rate adjusts continuously based on utilization error: `targetUtilization - currentUtilization`
- The adaptive rate (`rateAtTarget`) compounds at speed `ADJUSTMENT_SPEED * error` per second
- Initial `rateAtTarget` = 4% APR (`INITIAL_RATE_AT_TARGET`)
- Target utilization = 66.67% (`TARGET_UTILIZATION = 2/3 in WAD`)

Oracle Safety

- The maximum allowed delta between fast and slow oracle ticks = 953 ticks (~10% price move)
- During liquidation, current tick must be within 513 ticks of the TWAP (~5% deviation)
- If oracle deltas exceed thresholds, the protocol enters safe mode using more conservative price estimates
- The median tick buffer can only be updated if sufficient time has elapsed since the last observation

PanopticPool

Entry Points

- All user actions must originate from `dispatch()` or `dispatchFrom()` entry points
- `dispatchFrom()` requires the caller to have operator approval from the account owner
- Each dispatch call can execute exactly multiple action (mint, burn, mint, mint, burn etc.) and allows transiently insolvent states

Position Management

- Users can mint up to `MAX_OPEN_LEGS = 33` position legs total across all their positions
- Commission is split between protocol and builder (if builder code present) according to `PROTOCOL_SPLIT` and `BUILDER_SPLIT`
- Option sellers must pay back exact `shortAmounts` of tokens when positions are burned
- Option buyers must add back exact liquidity amounts that were removed when positions are burned

Solvency Requirements

- Users must remain solvent (per RiskEngine) after any mint, burn, or withdrawal operation
- Solvency is checked at the fast oracle tick, or at several oracle ticks if they diverge beyond the threshold

Premium Settlement

- Option sellers should not receive unsettled premium (premium not yet collected from Uniswap or paid by long holders)
- Sellers' share of settled premium must be proportional to their share of liquidity in the chunk
- Premium distribution ratio: `min(settled / owed, 1)` applies to all sellers in a chunk
- Long premium can be settled against solvent buyers to force payment

Liquidations

- Liquidations can only occur when `RiskEngine.isAccountSolvent()` returns false at the oracle tick

- The liquidator must close all positions held by the liquidatee
- Liquidation bonus paid to liquidator must not exceed the liquidatee's pre-liquidation collateral balance
- If the liquidation results in protocol loss, shares are minted to the liquidator to cover the difference
- It is acceptable for protocol loss to occur even if the liquidatee has residual token balance, if that balance is insufficient when converted

Premium Haircutting Invariant

- If premium is paid to sellers during a liquidation AND protocol loss exists after the liquidation, the premium must be haircut (clawed back) to reduce protocol loss
- After haircutting, protocol loss must be minimized but may still be positive if premium clawback is insufficient

Position Size Limits

- Individual position sizes are limited by the available liquidity in the Uniswap pool
- The maximum spread ratio (removed/net liquidity) is capped at `MAX_SPREAD = 90_000 / 100_000 = 90%`
- Position sizes must not cause integer overflows in any token amount or liquidity calculations

Cross-Contract Invariants

Oracle Consistency

- The oracle tick used for solvency checks must come from PanopticPool's oracle management
- All operations in a single transaction must use consistent oracle tick(s)
- The oracle must account for volatility via EMA and median filters to prevent manipulation

All trusted roles in the protocol

Role	Description
Guardian	Can only interact with the protocol through the **RiskEngine.sol** smart contract to: - Lock/unlock the pool (sets safeMode to 3) - Collect the outstanding protocol fees in the RiskEngine - The owner of BuilderFactory and can deploy new builderWallets

Running tests

Building

The traditional `forge` build command will install the relevant dependencies and build the project:

```
forge build
```

Tests

The following command can be issued to execute all tests within the repository:

```
forge test
```

Wardens are expected to populate the foundry.toml file with their own `eth_rpc_url` and `sepolia` infura endpoints.

Submission PoCs

Wardens are instructed to utilize the test suite of the project to illustrate the vulnerabilities they identify.

If a custom configuration is desired, wardens are advised to create their own PoC file that should be executable within the `test` subfolder of this contest.

All PoCs must adhere to the following guidelines:

- The PoC should execute successfully
- The PoC must not mock any contract-initiated calls
- The PoC must not utilize any mock contracts in place of actual in-scope implementations

Miscellaneous

Employees of Panoptic and employees' family members are ineligible to participate in this audit.

Code4rena's rules cannot be overridden by the contents of this README. In case of doubt, please check with C4 staff.